Automatic Financial Alert Generation from News Text and Chart Images

Jorge Carnicero Príncipe
Universidad Pontificia Comillas

Liam Esgueva González
Universidad Pontificia Comillas

Andrés Gil Vicente
Universidad Pontificia Comillas

Jorge González Pérez
Universidad Pontificia Comillas

Abstract

We present a multimodal pipeline for automatic alert generation from financial news. Given a news article and an optional chart image, the system extracts named entities and classifies the overall sentiment, generates a narrative description of the chart when one is present, and combines all these signals to produce a concise, context-specific alert. The pipeline is structured in three increasing levels: separate BiLSTM models for Named Entity Recognition (NER) and Sentiment Analysis (SA), a joint shared-encoder architecture that optimises both tasks simultaneously, and an advanced image-aware stage that integrates chart understanding via the LLMs DePlot and Qwen2.5. Alerts can be generated either by a deterministic rule-based module (intermediate level) or by the same large language model used for captioning (advanced level). Our experiments suggest that the joint model is more consistent than the independent NER and SA baselines. We have also analysed the sensitivity of the system to the loss weighting parameter and the prompt design for the alert generation, which has a huge impact on the results.

1 Introduction

Financial institutions, regulators, and media organisations must monitor large volumes of news in real time and react quickly to events that may signal reputational, economic, or geopolitical risk. Manual monitoring is slow and difficult to scale, while keyword-based systems lack the semantic understanding needed to identify the entities involved, interpret the tone of the content, and summarise the most relevant information in an actionable form.

The goal of this project is to build an automatic alert generation system that processes financial news in a multimodal setting. Given a news article and its associated chart image, the system extracts the key entities mentioned in the text, estimates the overall sentiment of the whole article, generates a textual description of the visual content, and combines all these signals into a descriptive, structured alert.

In this work, we develop the full pipeline from the ground up using some of the Deep Learning and Natural Language Processing techniques covered during the course. The system comprises four main components:

1. **Image Captioning:** generating a textual description of the chart image so that visual information can be incorporated into the final decision process, relying on DePlot as the core large language model for chart understanding, and then Qwen2.5 uses that output as the input to generate the caption.
2. **Named Entity Recognition (NER):** identifying persons, organisations, locations and dates mentioned in the text using a custom-trained BiLSTM-based model.

3. **Sentiment Analysis (SA):** classifying the overall tone of the text of the news article as positive, negative, or neutral with a second BiLSTM-based model.
4. **Alert Generation (AG):** combining the outputs of image captioning, NER, and SA, we use Qwen2.5 to produce an alert that references the detected entities and reflects the sentiment of the input news article.

Our implementation follows the three-level structure defined in the project specification and reaches the most sophisticated setting, the advanced level. The basic level consists of separate NER and SA models together with a rule-based alert generator. The intermediate level introduces a joint shared-encoder architecture for NER and SA, with its own joint loss function, as well as an LLM-based alert generation module. Finally, the advanced level extends the system to the multimodal setting by incorporating image captioning through a two-stage chart understanding pipeline.

The rest of this paper is organised as follows. Section 2 explains the related work we have used to inspire our model and architecture. Section 3 details the architecture of each component in the pipeline. Section 4 describes the datasets and the preprocessing challenges we encountered. Section 5 presents the experimental setup. Section 6 shows some alert results. Section 7 discusses the behaviour of the system, its strengths, and its limitations. Finally, Section 8 summarises the main findings and outlines future work.

2 Related Work

NER and SA in the financial domain. Most general-purpose NER models are trained on news corpora such as CoNLL-2003 (1), but financial text has domain-specific vocabulary and entity distributions. FiNER-ORD (2) provides the first large-scale, manually annotated NER dataset for financial news, which we use as our primary source. For sentiment analysis, FinBERT (3) is the reference model; we use it to generate pseudo-labels for sentences that do not have human-annotated sentiment.

Joint NER and SA. Multi-task learning with shared encoders often improves performance on related NLP tasks (5). Following the principle of joint many-task models (6)—where low-level token extraction provides strong inductive biases for high-level sentence classification—our intermediate pipeline employs a shared BiLSTM architecture. These shared semantic representations mutually regularize both tasks, yielding more robust adaptation to the financial domain.

Image captioning. Image captioning is commonly framed as an encoder–decoder task, where a visual encoder extracts image features and a text decoder generates a natural-language description (7). For chart images, however, the challenge is not only visual recognition but also faithful recovery of the underlying numerical structure. Recent approaches such as DePlot (8) address this by converting charts into linearised tables, which can then be used by a language model to produce more accurate textual descriptions.

Alert generation. Generating structured text from extracted facts is a well-known task in natural language generation from structured data, often referred to as data-to-text generation (10). We experiment with both rule-based templates and a prompted language model, Qwen2.5-1.5B-Instruct (9), as generators. By restricting the LLM input to the structured outputs (entities, sentiment, and caption) rather than the raw text, we aim to produce highly controlled alerts while minimising hallucinations.

3 Model Architecture

3.1 BiLSTM-CRF for NER (Separate Model)

Our separate NER model is a two-layer BiLSTM-CRF architecture. Tokens are represented with 100-dimensional pretrained GloVe embeddings, and are regularized with layer normalization and dropout. The encoder is composed of two stacked bidirectional LSTM layers with hidden size 256 per direction. We add a residual connection between the two BiLSTM blocks, followed by output normalization and dropout, before applying a linear layer that produces the scores over the BIO tags.

The model is trained by minimizing the CRF negative log-likelihood, and inference is carried out with Viterbi decoding. We train the model with a learning rate of 0.003, gradient clipping at 5.0, and patience-based learning-rate decay.

3.2 BiLSTM with Attention for SA (Separate Model)

Unlike the NER component, which performs token-level sequence labelling, the separate SA model is formulated as a sentence-level classification task. Input sentences are also encoded using 100-dimensional GloVe embeddings followed by a two-layer bidirectional LSTM with hidden size 256 per direction. But instead of using only the last hidden state, the model applies an attention mechanism over all contextualized token representations in order to identify the parts of the sentence that are most relevant for sentiment.

The resulting attention-weighted sentence vector is given to a linear classifier that predicts one of three classes: positive, negative, or neutral.

3.3 Joint Shared-Encoder (Intermediate Level)

Apart from the two separate models described above, we also implement an intermediate multi-task architecture in which NER and SA share a single encoder. Instead of training two independent recurrent models, this version uses one two-layer bidirectional LSTM to produce a common sequence of contextualized token representations, from which two task-specific prediction heads are derived.

Our NER branch applies a token-level linear classifier to the shared hidden states, while the SA branch uses an attention mechanism to aggregate the same sequence of hidden representations into a sentence-level vector. Then, this is mapped to the three sentiment classes. In this way, the model learns these shared representations that can support both entity labelling and sentiment classification.

The training objective combines the losses of both tasks:

$$\mathcal{L} = \lambda_{\text{NER}}\,\mathcal{L}_{\text{NER}} + \lambda_{\text{SA}}\,\mathcal{L}_{\text{SA}}, \tag{1}$$

where $\mathcal{L}_{\text{NER}}$ is token-level cross-entropy with padding positions ignored, and $\mathcal{L}_{\text{SA}}$ is sentence-level cross-entropy over the sentiment labels.

At inference time, both outputs are obtained from a single forward pass through the shared encoder, making the joint model more efficient than running the two separate architectures sequentially. We have implemented the whole pipeline in a way that allows the user to use a flag that enables this shared setting or deactivate it and just use both separate models depending on what is preferred.

3.4 Image Captioning (Advanced Level)

CNN + LSTM captioner (experimental). Our first approach to image captioning was a custom encoder-decoder model trained on a Hugging Face dataset of chart–explanation pairs. The encoder was a CNN-based visual backbone (ResNet-50) and the decoder was an LSTM trained to generate captions autoregressively. Although the model reached a reasonably low training loss when fine-tuning the last layers of the CNN and the LSTM from scratch (approximately 0.9), its generalisation to unseen chart images was poor. The generated captions were usually well-formed syntactically, but often failed to describe the actual chart content correctly. This made the model unreliable for downstream alert generation. Moreover, this approach also had the disadvantage of taking too long to train (almost 4 hours).

DePlot + Qwen2.5. Because the custom captioner did not provide sufficiently descriptive captions, we switched to the alternative strategy suggested in the project statement, which consisted in using a pretrained captioning pipeline. After doing some research and trying a few different Hugging Face models, our final approach combines DePlot (8), which extracts a linearised table from the information shown in the chart image, with Qwen2.5-1.5B-Instruct, which generates a caption from that table. In practice, this approach produced clearly more useful and semantically accurate captions than our manual CNN+LSTM model. That is also why we chose this approach as the image-captioning component in the final pipeline.

3.5 Alert Generation

Our system supports different execution modes that can be selected according to the user's needs through a set of configuration flags described in the Quickstart_README file.

Rule-based generator. This naive approach, maps (entities, sentiment) pairs to alert templates with a deterministic algorithm. For example, if an ORG entity is detected with negative sentiment, the output is *"REPUTATION RISK: [Org] mentioned NEGATIVELY."*. The full decision logic covers several combinations of entity types and sentiment classes, generating a neutral or just informative alert if no clear signal is found. This rule-based mode is available when an image caption is available, but also when there is no image in the article.

LLM-based generator. When the llm mode is selected, the rule-based alert is replaced by a generation that comes from the Hugging Face model Qwen2.5. This model receives a structured input consisting of: the detected entities (with their corresponding types), the sentiment class, the confidence score, and the image caption if one was produced. The system prompt we have selected calibrates the certainty of the alert to the confidence score of the predicted sentiment. When this score is high, the generated alert tends to reflect fidelity in relation with that sentiment, but when the score is not considered good enough, the alert transmits that uncertainty. Moreover, if the LLM call fails for any reason, the system falls back to the rule-based generator transparently.

We experimented with two input configurations: giving the model the full original news sentence alongside the structured outputs, and giving it only the structured outputs (ner entities + sentiment + caption). We observed that restricting the input just to structured data without the original article text kept the alert more focused on our SA and NER predictions.

We finally decided to follow this approach instead of using the initial raw text as another input. When we tried adding the original text as another input, we observed that the alert was very biased towards this text, almost ignoring the SA and NER predictions.

4 Data and Preprocessing

4.1 Datasets

FiNER-ORD. We use the `gtfintechlab/finer-ord` dataset from HuggingFace (2). It contains financial news sentences annotated with IOB2 tags for PER (persons), LOC (locations) and ORG (organisations). We extend the tag set with YEAR_B/YEAR_I labels, which are assigned automatically by a regular-expression post-processor to any four-digit year token, because the real dataset does not have a tag for YEAR.

Silver sentiment labels. FiNER-ORD does not include sentiment annotations. We pass all sentences through FinBERT (`ProsusAI/finbert`), a BERT model fine-tuned on Financial PhraseBank (3), to obtain silver positive/negative/neutral labels. The median confidence of these pseudo-labels lies between 87% and 92%, which we consider reliable enough for training. This lets us train both NER and SA on the same in-domain corpus without needing a second dataset.

ChartQA-X. For our experimental custom image captioning model (CNN+LSTM), we utilised the `shamanthakhegde/ChartQA-X` dataset from Hugging Face (11). This dataset contains a diverse collection of chart images paired with an explanation.

GloVe embeddings. Both BiLSTM models are initialised with 100-dimensional GloVe vectors (4) pretrained on Wikipedia and Gigaword. Words absent from the vocabulary are mapped to a shared `<UNK>` embedding.

4.2 Preprocessing Challenges

During dataset construction we identified three issues that had a measurable impact on training and evaluation quality.

Wrong BIO tag ordering. In an initial version of the configuration file, the NER tag indices were listed in a different order than the one used internally by the model (e.g., LOC_B and ORG_B were swapped). As a result, the model was learning to label organisations as locations and vice versa. The separate NER model was trained with this bug, which explains its near-zero entity-level F1 on the test set (Section 5). The joint model was trained after the fix and achieves substantially better results.

Duplicate and noisy sentences. The dataset is built from web-scraped financial news. We found a significant number of duplicate sentences, identical strings appearing in both the training and test splits, which artificially inflates evaluation scores by allowing the model to memorise seen examples. We also found uninformative strings such as *"loading page..."* or bare URLs that carry no financial information. Both categories were removed before building the final dataloaders.

Truncated multi-word entities. The sentence splitter occasionally cuts a multi-word entity across a boundary. For example, *"Palo Alto"* might end one sentence while *"Networks"* begins the next, tagged as `ORG-I` with no preceding `ORG-B`. Without the preceding context, that label is incorrect and confuses the model. We strip dangling `-I` tags with no matching `-B` during preprocessing.

5 Experiments

5.1 Ablation Study: Loss Weighting

To understand the trade-off between NER and SA performance in the joint model, we trained five variants with different values of λ_{NER} (and $\lambda_{\text{SA}} = 1 - \lambda_{\text{NER}}$). Results are shown in Table 1.

Table 1: Effect of the loss weighting parameter on NER and SA performance.

λ_{NER}	**NER F1**	**SA F1**
0.3	0.702	0.665
0.5	0.694	0.692
0.6	0.687	0.679
0.7	0.698	0.686
1.0	0.705	0.283

Setting $\lambda_{\text{NER}} = 1.0$ effectively disables the SA loss and degrades SA F1 to 0.283 while achieving the best NER score (0.705), confirming that the two tasks do share useful representations. At the other extreme, $\lambda_{\text{NER}} = 0.3$ yields the strongest SA score (0.692) at a small cost to NER (0.702). We chose $\lambda_{\text{NER}} = 0.6$ as our default, which offers a balanced trade-off without sacrificing either task significantly. Figure 4 (see chart 3 in Annex) shows the TensorBoard curves for total training loss and average validation F1 across the main joint-model runs, illustrating the convergence behaviour of the different configurations that we have been testing in order to obtain the best possible performance.

5.2 Image Captioning Experiments

As noted in Section 3, we evaluated two different approaches for chart image captioning: our custom CNN+LSTM architecture and the pretrained DePlot+Qwen pipeline.

CNN+LSTM experiments. While the CNN+LSTM model converged during training and achieved a low training loss, manual inspection of its outputs revealed a severe limitation.

The following examples illustrate the performance gap between the human explanations and the captions generated by our CNN+LSTM model. The charts images are provided in Annex A.

Example 1 (See Chart 1 in Annex)	**Example 2** (See Chart 2 in Annex)
Target explanation The line graph shows a steady increase in the **Chained Consumer Price Index** over time, with the highest point occurring in **May '21**. The data points are marked with blue dots and connected by a blue line, indicating the trend of the index over the months.	**Target explanation** To determine the infant mortality rate in the Netherlands in 2019, we refer to the graph. By locating **2019** on the x-axis, the corresponding value is **3.6 per 1,000 live born children**.
Generated explanation — CNN+LSTM The line graph shows a steady increase in the **gross domestic product** from **May 2019 to May 2021**. The highest point is described inconsistently, and the output contains `<UNK>`.	**Generated explanation — CNN+LSTM** The line graph describes average usual weekly hours worked from **2008 to 2019**. It incorrectly reports a value of **10.1** and identifies the wrong variable.

Table 2: Side-by-side comparison of error patterns in Example 1 and Example 2.

Table 2 illustrates two representative examples of these failures. The architecture demonstrates a baseline capacity for visual feature extraction, sometimes successfully identifying temporal axes (e.g., the 2008–2019 range in Example 2) and general trend directions (e.g., the "steady increase" in Example 1). However, it suffers from significant hallucinations, consistently misidentifying financial indicators—such as confusing price indices with GDP—and generating factually incorrect data points (e.g., predicting 10.1 instead of 3.6). This inability to align visual features with the actual chart metadata makes the custom CNN+LSTM approach unreliable for a critical financial alert.

6 Results

6.1 Evaluation Metrics

We report F1-score as the main evaluation metric for both NER and SA, since accuracy is not informative enough under the strong class imbalance present in our data. In the NER setting, most tokens belong to the non-entity class `O`; in the SA setting, neutral examples are the most frequent. Therefore, a model could obtain high accuracy by mostly predicting the majority class, while still failing to recognise relevant entities or minority sentiment classes.

For NER, we use entity-level micro-F1 computed with `seqeval`. This metric only counts an entity as correct when both its boundaries and its type are predicted exactly. For SA, we use macro-F1, which averages performance across the positive, negative and neutral classes and therefore gives the same importance to each class regardless of its frequency.

6.2 Quantitative Results: Separate vs. Joint Models

Separate NER failure. The standalone NER model achieves a micro-F1 of 0.003, which is effectively zero. This is a direct consequence of the BIO tag ordering bug described in Section 4: the model was trained mapping entity labels to the wrong indices, so its predictions are systematically incorrect when evaluated with the correct mapping. The bug was discovered during post-hoc analysis of the confusion matrix, which showed that organisation and location predictions were perfectly swapped. Table 3 summarises the test-set results for all configurations.

Table 3: Test-set results for the separate and joint models.

Model	**NER F1**	**SA F1**	**SA acc.**
Separate BiLSTM-NER	0.003	—	—
Separate BiLSTM-SA	—	0.586	0.638
Joint encoder (NER head)	0.687	—	—
Joint encoder (SA head)	—	0.679	0.713

Joint model gains. After fixing the tag ordering and retraining within the joint architecture, the NER head achieves a micro-F1 of 0.687 ($\Delta = +0.684$ relative to the separate model). Per-entity results are: PER 0.765, LOC 0.704, ORG 0.585. The lower score for organisations is consistent with findings in the literature, as organisation names are more varied and context-dependent than person or location names.

The joint model also improves SA F1 from 0.586 to 0.679 ($\Delta = +0.069$), with consistent gains across all three classes. Table 4 shows these results.

Table 4: Per-class F1 for sentiment analysis.

Model	Positive	Negative	Neutral
Separate	0.534	0.522	0.702
Joint	0.611	0.629	0.796

6.3 Qualitative Analysis: Alert Generation Examples

Below we show representative outputs from the two alert modes for this input:

News: "For the first quarter of their 2026 fiscal year, Apple reported a revenue of 143.76 billion U.S. dollars. This was the highest quarterly revenue of the company."

Image:

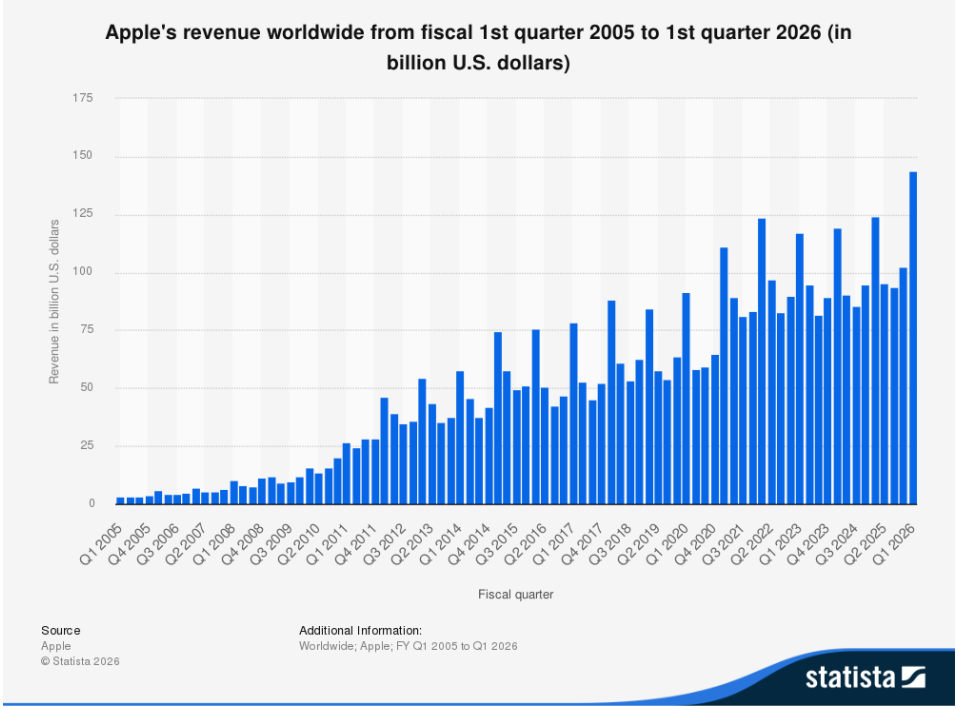


Figure 1: Input chart used for multimodal alert generation (sample p1).

Chart caption: "The chart titled 'Apple's revenue worldwide from fiscal 1st quarter 2005 to 1st quarter 2026' illustrates Apple's global revenue over time, showing an overall upward trend."

Entities: 2026 [YEAR], Apple [ORG] *Sentiment:* positive (0.9536)

Example 1 — LLM mode

Alert: "Apple's revenue worldwide shows steady growth from fiscal 2005 to 2026, indicating a positive trend despite the uncertainty surrounding future projections."

Example 2 — Rule-based mode

Alert: "POSITIVE UPDATE: Apple reported POSITIVELY. [Image context: The chart titled 'Apple's revenue worldwide from fiscal 1st quarter 2005 to 1st ...]"

7 Analysis

7.1 Effect of Shared Representations

Beyond the tag-ordering bug, our analysis suggests that the shared encoder benefits from the sentiment signal: phrases that carry strong sentiment (e.g., *"record losses"*, *"CEO resigns"*) tend to co-occur with named entities, and learning to classify sentiment implicitly encourages the encoder to attend to those mentions. This is a case where multi-task learning genuinely helps even though the two tasks share only the encoder, not the decoder.

7.2 Sensitivity of Alert Quality to Prompt Design

We ran several experiments varying the system prompt for the LLM-based alert generator. A prompt that allowed the model to rephrase the original sentence freely produced more fluent alerts but occasionally introduced facts not present in the input. Restricting the input to structured data only (entities + sentiment + caption) and adding explicit instructions to stay conservative when confidence is low significantly reduced hallucinations. We also observed that the confidence calibration rules in the prompt (hedging below 0.80, uncertainty below 0.55) produce noticeably different alert phrasings that correctly reflect the model's uncertainty.

7.3 Strengths

- The joint model delivers competitive NER and SA in a single forward pass, making it suitable for low-latency applications.
- The DePlot + Qwen captioning pipeline requires no fine-tuning and generalises well across different chart types (bar, line, pie).
- The rule-based fallback guarantees that an alert is always produced, even if the LLM is unavailable or returns an empty response.
- The system is configurable with different flags as we mentioned in Section 3.

7.4 Limitations and Future Work

- NER performance on ORG entities (F1 = 0.585) is notably lower than on PER and LOC. Organisation names in financial text are highly variable (ticker symbols, abbreviations, holding companies) and a CRF layer or a pre-trained financial language model like FinBERT could help.
- The silver sentiment labels from FinBERT introduce label noise. Training directly on Financial PhraseBank with human annotations, or fine-tuning on a curated subset, would likely improve SA quality.
- Alert generation quality is hard to evaluate automatically. We report qualitative examples, but a proper evaluation would require human judgements or a dedicated benchmark.
- Although we initially intended to deploy our custom-built CNN+LSTM captioner, it was outperformed by DePlot+Qwen by a large margin. To ensure the evaluation of the alert generation was not bottlenecked by captioning errors, we integrated these robust pre-trained models into the main pipeline.

8 Conclusion

We have built a complete multimodal pipeline for automatic financial alert generation that covers all three levels of the project specification. The main findings are:

1. A shared BiLSTM encoder trained jointly on NER and SA substantially outperforms two independently trained models, both in absolute terms and after correcting for a data-preprocessing bug that completely invalidated the standalone NER results.
2. The choice of loss weighting λ has a measurable but moderate impact on performance; $\lambda_{\text{NER}} = 0.6$ provides a good balance.
3. Combining DePlot (chart-to-table) with Qwen2.5 (table-to-text) produces high-quality chart captions without any task-specific fine-tuning.
4. LLM-based alert generation with a carefully designed system prompt produces more natural and informative alerts than rule-based templates, while the rule-based module provides a robust fallback.

Future work should focus on improving organisation NER with domain-adapted encoders, reducing silver-label noise in the SA training data, and incorporating human evaluation of alert quality into the experimental pipeline.

Team Declaration

Jorge Carnicero Príncipe and Jorge González Pérez led the development of the image captioning component. They initially designed and implemented a custom captioning architecture based on CNN and LSTM modules developed from scratch. This initial phase required an extensive search for suitable chart-to-text datasets on Hugging Face, followed by an exhaustive fine-tuning process to adapt the model to the charts. Although this approach was functional, it produced relatively weak captions and required substantial computational time and manual engineering effort. To ensure the highest quality for the downstream alert generation, they transitioned to a more effective pretrained approach. Consequently, they implemented the final image captioning pipeline based on `DePlot` and `Qwen2.5`, which was adopted in the final multimodal version of the system.

Liam Esgueva González and Andrés Gil Vicente led the dataset preparation and the development of the text understanding models. Since it was difficult to find a dataset containing both Named Entity Recognition (NER) and Sentiment Analysis (SA) annotations, they selected a financial dataset annotated only for NER and extended it with sentiment labels automatically. To do so, they used a pretrained transformer-based sentiment analysis model from Hugging Face to infer the sentiment of the news items in the dataset. As a result, the dataset moved from a gold-style resource for NER to a silver-label setting for sentiment, which was then used to construct the training, validation, and test splits. They also implemented the bidirectional LSTM models used for the NER and SA tasks.

Finally, all four team members contributed jointly to the final alert generation module. In particular, the team collaborated on integrating the outputs of the image captioning, NER, and SA components into a unified pipeline, using `Qwen2.5` to generate the final alert, and experimenting with different system prompts and input configurations in order to improve the quality, consistency, and reliability of the generated outputs.

References

[1] Tjong Kim Sang, E.F. and De Meulder, F. (2003). Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. *Proc. CoNLL*.

[2] Shah, R. et al. (2023). FiNER-ORD: Financial Named Entity Recognition. `https://huggingface.co/datasets/gtfintechlab/finer-ord-bio`

[3] Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. *arXiv:1908.10063*.

[4] Pennington, J., Socher, R., and Manning, C. (2014). GloVe: Global Vectors for Word Representation. *Proc. EMNLP*.

[5] Caruana, R. (1997). Multitask Learning. *Machine Learning*, 28(1):41–75.

[6] Hashimoto, K., Xiong, C., Yoshikawa, Y., and Socher, R. (2017). A Joint Many-Task Model: Growing a Neural Network for Multiple NLP Tasks. *Proc. EMNLP*. https://arxiv.org/abs/1611.01587

[7] Vinyals, O. et al. (2015). Show and Tell: A Neural Image Caption Generator. *Proc. CVPR*.

[8] Liu, F. et al. (2023). DePlot: One-shot visual language reasoning by plot-to-table translation. *Proc. ACL Findings*.

[9] Qwen Team (2024). Qwen2.5: A Party of Foundation Language Models. https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

[10] Parikh, A.P. et al. (2020). ToTTo: A Controlled Table-To-Text Generation Dataset. *Proc. EMNLP*.

[11] Masry, A. et al. (2022). ChartQA: A Benchmark for Question Answering about Charts with Visual Logical Reasoning. *Proc. ACL Findings*. https://huggingface.co/datasets/shamanthakhegde/ChartQA-X

9 Use of Artificial Intelligence

Artificial intelligence tools were used throughout the development of this project as auxiliary support, without replacing the authors' own understanding, experimentation, implementation, or analysis. Their use was limited to tasks aimed at improving the quality, organisation, and presentation of both the report and the codebase:

- **Language and style refinement:** AI tools were used to detect and correct spelling, typographical, and punctuation errors, as well as to improve overall clarity, reduce repetition, and ensure a consistent academic tone throughout the report.
- **Report structure:** AI support was used to adapt the structure of a previous report template to the requirements of the current project, preserving consistency in formatting and style.
- **Code repository organisation:** AI tools helped to propose a clearer structure for the code repository, including the separation of model definitions, training scripts, preprocessing utilities, configuration files, and documentation.
- **Code debugging support:** AI was used to assist in identifying potential sources of errors, interpreting error messages, and suggesting possible fixes during the implementation and testing stages.
- **Code optimisation and readability:** AI tools provided suggestions to simplify parts of the code, remove redundant logic, improve function organisation, and make the implementation easier to maintain.

All final decisions regarding the methodology, experiments, implementation, results, and conclusions were made by the authors. The generated suggestions were critically reviewed, adapted, and validated before being incorporated into the final project.

A Visual Annex

This appendix contains the original chart images used in the qualitative evaluation of the image captioning module.

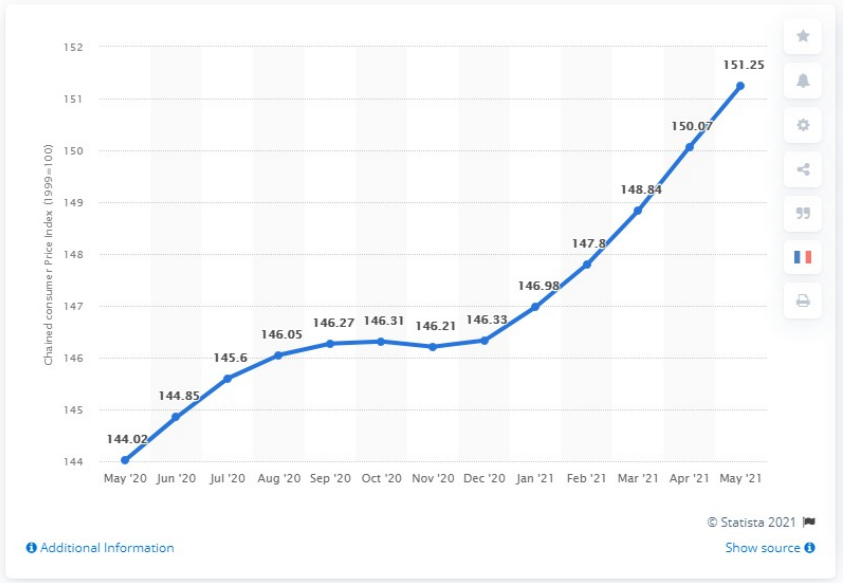


Figure 2: Chart corresponding to Example 1: Chained Consumer Price Index.

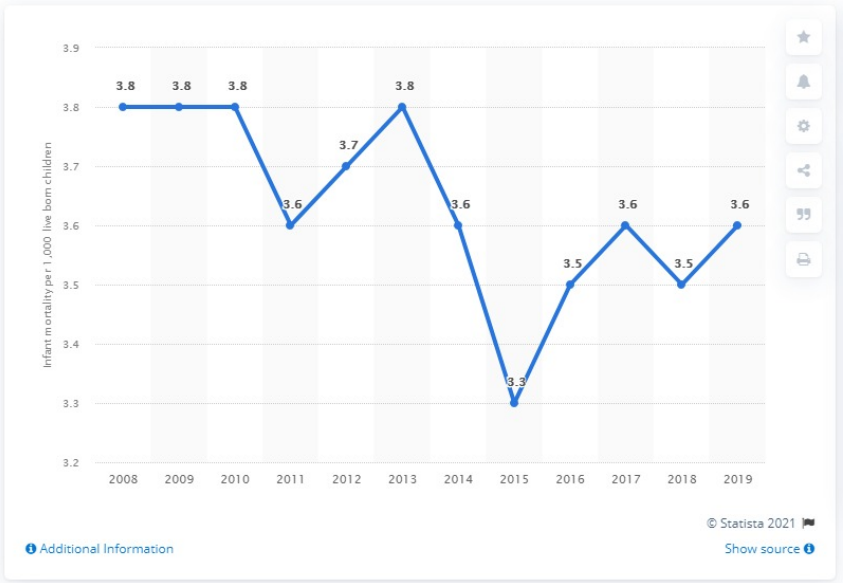


Figure 3: Chart corresponding to Example 2: Infant mortality rate in the Netherlands.

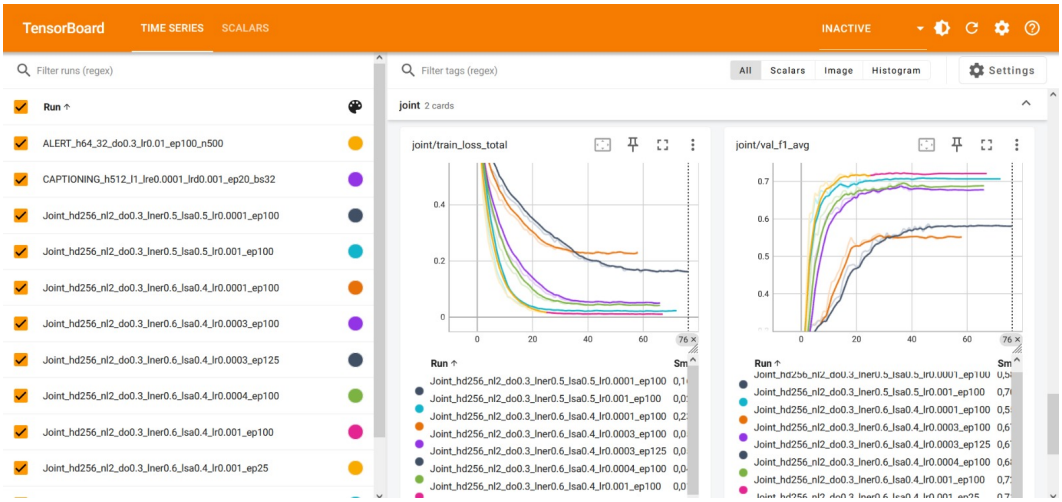


Figure 4: TensorBoard curves for the main joint-model runs: total training loss (left) and average validation F1 (right) under different hyperparameters